



Intents and Broadcast Receivers

IS2205 - Mobile Application Design & Development





Intents

“The most important property of a program is
whether it accomplishes the **intention** of its
user.”

— C.A.R. Hoare



Intent

- A “messenger” that allows to indicate the “intention” of requesting an action from a different component at runtime.
- Intent provides a facility for performing late runtime binding between the code in different applications as well.
- Intents can start activities, services and deliver broadcasts to broadcast receivers

Types of Intents in Android

There are two main intent categories:

Explicit: The intent which has exactly defined, one target

Implicit: The intent which specifies only an action and the component should be selected by the Android system

Also

Pending: An intent to be used in the future (usually by a different application).A description of an Intent and target action to perform with it.

Intent : Action

- An `Intent` usually has a set of primary information:
 - **Action** to be performed(`String`) : using the `setAction()`
 - An action in an Android intent is a string that describes the operation that the intent wants to perform. Some common intent actions:
 - `ACTION_VIEW`: Display the specified data to the user. i.e.: Image to be sent to Gallery
 - `ACTION_SEND`: Send the specified data to another app. i.e: social media share
 - `ACTION_PICK`: Let the user pick a piece of data, such as a contact, image, or file.
 - `ACTION_EDIT`: Let the user edit the specified data.
 - `ACTION_MAIN`: Launch the main activity of the app.
 - `ACTION_SEARCH`: Perform a search.

Quiz in VLE

Intent : Category

- **Category** in intents are used to classify intents and to help the system determine which apps to match an intent to. i.e.: the category `CATEGORY_LAUNCHER` is used to identify intents that can be used to launch a new app. Some examples:
 - `CATEGORY_APP_BROWSER`: Identify intents that can be used to open a web browser. It is typically used with the `ACTION_VIEW` action.
 - `CATEGORY_APP_CALCULATOR`: This category is used to identify intents that can be used to open a calculator app.
 - `CATEGORY_APP_CALENDAR`: This category is used to identify intents that can be used to open a calendar app.
 - `CATEGORY_ALTERNATIVE`: This category is used to identify intents that can be used to perform an alternative operation. For example, if an app does not support the `ACTION_VIEW` action, it can declare an intent filter with the `ACTION_EDIT` action and the `CATEGORY_ALTERNATIVE` category.

Intent : Type

- MIME type may specify the type of data that the intent delivers.
- **Type** - MIME type is set by setType() method
 - text/plain - Plain text
 - text/html - HTML
 - text/css - CSS
 - image/jpeg - JPEG image
 - audio/mpeg - MP3 audio
 - video/mp4 - MP4 video
 - application/pdf - PDF document
 - application/zip - ZIP file

Intent : Type...

Sending an image via an Intent

```
val intent = Intent(Intent.ACTION_SEND)
intent.type = "image/png" // MIME type
intent.putExtra(Intent.EXTRA_STREAM, imageUri)
startActivity(Intent.createChooser(intent, "Share Image"))
//chooser forces the user to select an app
```

Intent : Component

- **Component** specifies an explicit name of a component class to use for the intent.
 - Normally this is determined by looking at the other information in the intent.(Action,Category...)
 - By specifying this attribute, all of the other resolution attributes become optional.

```
val intent = Intent()  
intent.setComponent(ComponentName("com.example.myapp",  
    "com.example.myapp.MainActivity"))  
// Start the activity by intent  
startActivity(intent)
```

Intent : Flag

- **Flag** – A hint to the Android system for the how intent should be handled
 - `FLAG_ACTIVITY_NEW_TASK`: The activity has to be launched in a new task.
 - `FLAG_ACTIVITY_CLEAR_TOP`: Launches a new task, and all other activities in the current task are finished.
 - `FLAG_ACTIVITY_SINGLE_TOP`: Activity will be launched in the existing task if it is already running at the top of the task.
 - `FLAG_ACTIVITY_NO_HISTORY`: This flag causes the activity to not be added to the task history.

Activity : Flag Parameter

Suggest a use-case for the each of the following flags in an Android application

- **FLAG_ACTIVITY_NEW_TASK:** The activity has to be launched in a new task.
- **FLAG_ACTIVITY_CLEAR_TOP:** Launches a new task, and all other activities in the current task are finished.
- **FLAG_ACTIVITY_SINGLE_TOP:** Activity will be launched in the existing task if it is already running at the top of the task.
- **FLAG_ACTIVITY_NO_HISTORY:** This flag causes the activity to not be added to the task history.

Quiz in VLE

Intent : extras

- **Extras** - This is a collection of any additional information.
- To add an extra, you use the `putExtra()` method.
- To retrieve an extra from an intent, you use the `getStringExtra()` method.
(`getIntExtra`, `getBooleanExtra`)

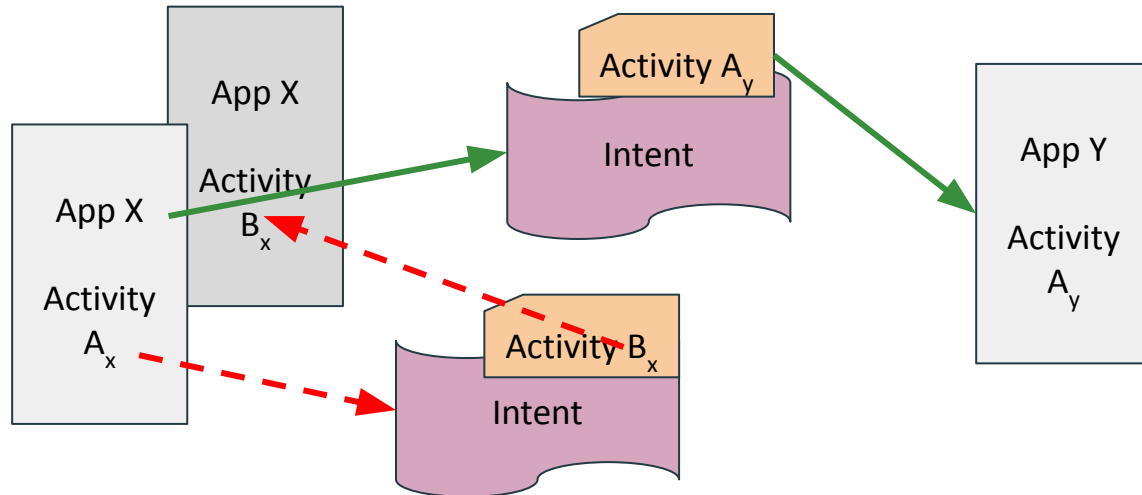
```
// An intent to start the MyActivity
val intent = Intent(this, MyActivity::class.java)
// Add an extra to the intent
intent.putExtra("my_string_extra", "This is my
string extra")
// Start the activity
startActivity(intent)
```

```
// Get the intent inside the activity
val intent = getIntent()
```

```
// Get the extra from the intent
val myStringExtra =
intent.getStringExtra("my_string_e
xtra")
```

Explicit intent

- Fulfills a request **using a specific component**
- Navigates internally to an Activity in your app
- Navigates to a specific third-party app or another app you've written



Explicit intent examples

Navigate between activities in your app:

```
fun viewNoteDetail() {  
    val intent = Intent(this, NoteDetailActivity::class.java)  
    intent.putExtra(NOTE_ID, note.id)  
    startActivity(intent)  
}
```

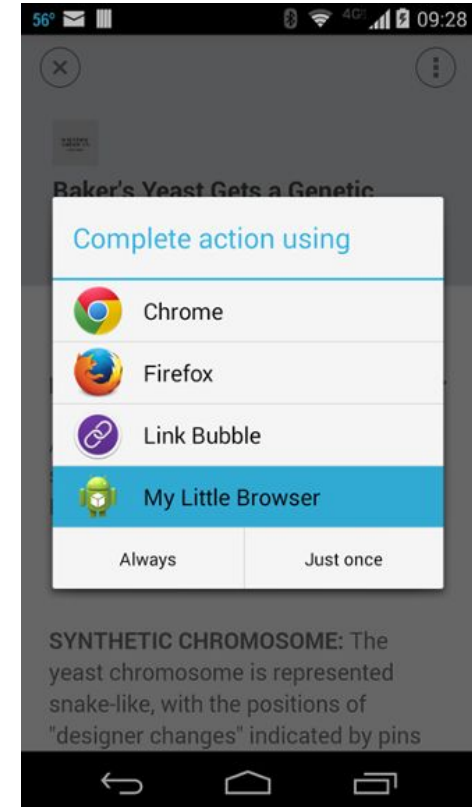
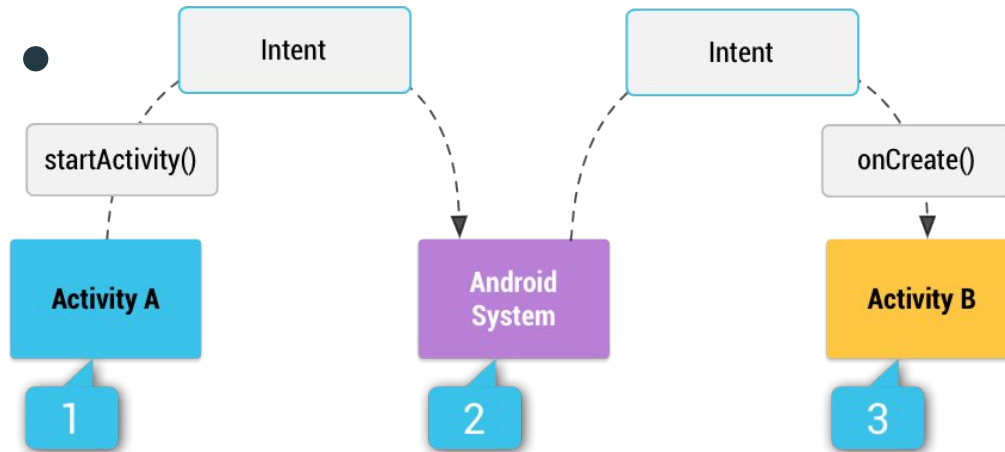
Navigate to a specific external app:

```
fun openExternalApp() {  
    val intent = Intent("com.example.workapp.FILE_OPEN")  
    if (intent.resolveActivity(packageManager) != null) {  
        //runtime test  
        startActivity(intent) //the availability of the activity  
    }  
}
```

Implicit intent

- Provides generic action the app can perform
- Resolved using mapping of the data type and action to known components
- Allows any app that matches the criteria to handle the request

Implicit intent Resolution



Implicit intent example

```
fun sendEmail() {  
    val intent = Intent(Intent.ACTION_SEND)//implicit  
    intent.type = "text/plain"  
    intent.putExtra(Intent.EXTRA_EMAIL, emailAddresses)  
    intent.putExtra(Intent.EXTRA_TEXT, "Hey diddle diddle")  
  
    if (intent.resolveActivity(packageManager) != null) {  
        startActivity(intent)  
    }  
}
```

Pending intents

- A `PendingIntent` is a reference to a token maintained by the Android system.
- “Application A can pass a `PendingIntent` to application B in order to allow application B to execute predefined actions on behalf of application A; regardless of whether application A is still alive.”

Pending intents...

- A typical use-case of a `PendingIntent` is to launch an application by clicking on the notification screen.
- A pending intent may be created to show a notification for a new message by a messenger app.
- If clicked, the app will be launched and the message will be directly shown.

Exercise

Create an intent (with Androids common) an Action, a Category and a MIME type and use to launch an activity.

Define (a) suitable intent filters to route your intent to the specific activity

Intent Filters

What are Intent Filters?

Declare which intents a component can respond to

Usually defined in `AndroidManifest.xml`

Enable handling of **implicit intents** by matching intent properties

```
<intent-filter>  
  <action android:name="android.intent.action.VIEW" />  
  <category android:name="android.intent.category.DEFAULT" />  
</intent-filter>
```

Intent Filters...

Components of Intent Filters

- **Action:** Defines what to do (e.g., VIEW, SEND)
- **Category:** Adds context (e.g., DEFAULT, BROWSABLE)
- **Data:** Specifies the type of data (MIME type, URI scheme)

```
<intent-filter>  
  <action android:name="android.intent.action.VIEW" />  
  <category android:name="android.intent.category.BROWSABLE" />  
  <data android:scheme="http" />  
</intent-filter>
```

Intent Filters...

Rules for the Matchmaker

Component(i f)	Matching Rule	Notes
Action(1 N)	Intent's action must exactly match one declared in the filter	Mandatory; no partial or subset match allowed. setAction : method
Category(N N)	All categories in intent must be declared in the filter (intent categories $\subseteq$ filter categories)	Filter can declare more categories than intent has. addCategory
Data(1 N)	If filter declares data, intent data must match scheme, host, path, and/or MIME type	Optional; if absent in filter, and intent may match as above

Intent Filters...

Data and Type

```
// The setDataAndType() : Kotlin
// second set erases the first
val i = Intent(Intent.ACTION_SEND)
i.setData(imageUri) // sets data, but clears any existing type
i.setType("image/png") // sets the type, but overwrites the data URI

// Setting both together
val intent = Intent(Intent.ACTION_SEND).apply {
    setDataAndType(imageUri, "image/png")
}
```

Intent Filters...

Data and Type

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:scheme="https" android:host="www.example.com"
    android:pathPrefix="/profile" />
    <data android:mimeType="application/json" android:scheme="content" />
</intent-filter>
```

```
//Kotlin : Match due to data elements of first definition
val intent = Intent(Intent.ACTION_VIEW).apply {
    data = Uri.parse("https://www.example.com/profile/saman")
}
```

Intent Filters...

Data and Type

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:scheme="https" android:host="www.example.com"
        android:pathPrefix="/profile" />
    <data android:mimeType="application/json" android:scheme="content" />
</intent-filter>
```

```
//Kotlin : Match due to data elements of second definition
val i = Intent(Intent.ACTION_VIEW).apply {
    setDataAndType(Uri.parse("content://media/external/file/12"),
        "application/json")
}
```



Broadcast Receivers

Some asteroids have us in their sights. Be nice to sort of go near them and find out what they're made of, possibly tag their ears so they're always broadcasting to us their location. In case one of their trajectories head straight for us, we'll know well in advance to do something about it.

- Neil deGrasse Tyson

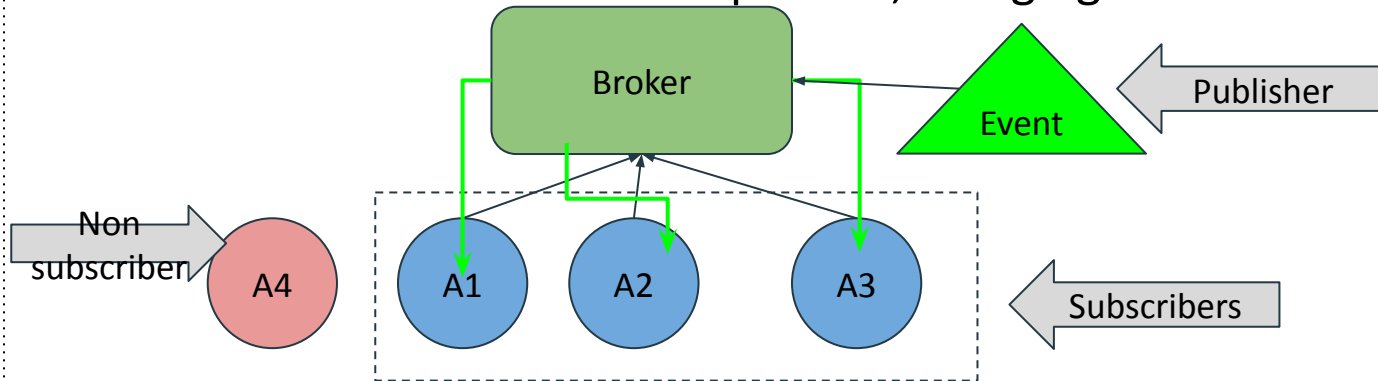


What is pub-sub?



Broadcasts in Android

- Android apps can send or receive broadcasts from:
 - Android system
 - Other Android apps(custom broadcasts)
- A **publish-subscribe** design.
- Android OS broadcast bootup event, charging started...



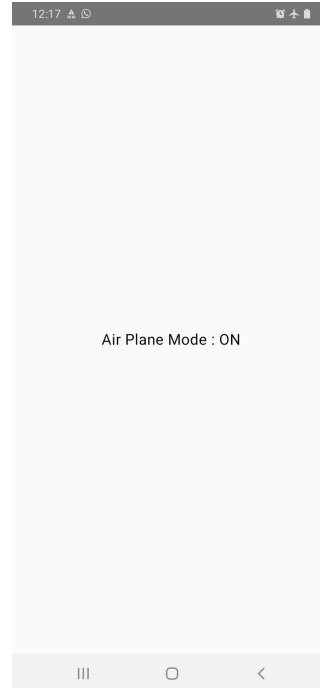
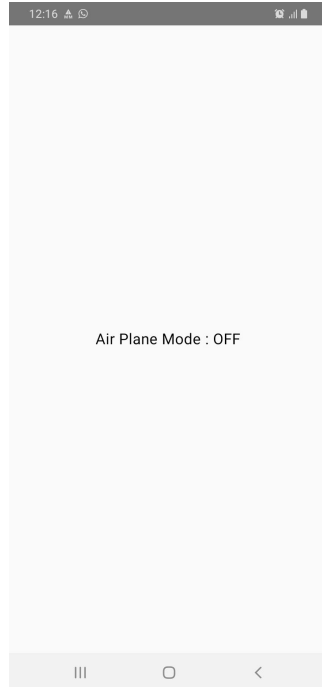
Exercises

Read the Android documentation and create a simple Android Application with a broadcast receiver component.

Implement a log message to identify the receiving of the broadcast.

Discuss the Sample application.

Broadcast Receiver Example



Revision Exercises

Draw a diagram to explain the functionality of AirPlaneModeBCast application with its main components such as Activity, ViewModel and BroadcastReceiver.

Thank you!